

Mobile Application Development

Complete Documentation with React Native Concentration

Student Developer Roadmap

May 28, 2026

Contents

1	Mobile App Development Overview	3
1.1	Types of Mobile Apps	3
1.2	Why React Native?	3
2	Mobile App Architecture	3
3	Development Methods for Mobile Apps	4
4	Essential Tools for Mobile Development	4
4.1	React Native Development Environment	4
4.2	React Native Libraries	4
4.3	Backend for Mobile Apps	5
4.4	Testing & Debugging	5
4.5	Building & Distribution	5
5	Learning Resources	5
6	Project Sequence for Mobile Apps	6
6.1	Level 1 – Basic React Native Apps	6
6.2	Level 2 – Navigation & Multi-Screen Apps	6
6.3	Level 3 – Device Features Integration	6
6.4	Level 4 – Full-Stack Mobile Apps	7
6.5	Level 5 – Production-Ready Apps	7
7	Concentration: React Native Deep Dive	7
7.1	Core Components vs Web	7
7.2	Basic React Native Component	7
7.3	Navigation with React Navigation	9
7.4	API Calls from React Native	11
7.5	Device Features Integration	13
7.6	Location and Maps	14
7.7	Push Notifications with Expo	16
8	Building and Publishing	17
8.1	Expo Build Commands	17
8.2	App Store Requirements	18
9	Learning Plan for Mobile	18

1 Mobile App Development Overview

1.1 Types of Mobile Apps

Type	Description
Native App	Built for specific platform (iOS: Swift/Kotlin, Android: Kotlin/-Java). Best performance.
Cross-Platform	One codebase for both iOS & Android (React Native, Flutter). Recommended for most projects.
Hybrid App	Web technologies inside native container (Ionic, Cordova).
Progressive Web App (PWA)	Web app that works offline and can be installed.

1.2 Why React Native?

- Write once, run on iOS and Android
- Uses JavaScript and React (you already know this!)
- Native performance (not webview-based)
- Large community and ecosystem
- Hot reloading for fast development
- Backed by Meta (Facebook)

2 Mobile App Architecture

React Native App

JavaScript Code (React Components, State, Logic)

Bridge (Serializes JS calls to Native)

Native Modules (Camera, GPS, Push Notifications)

iOS (UIKit/Swift)

Android (Kotlin/Java)

Backend API (Node.js + Express)

React Native + Expo

MongoDB / PostgreSQL

3 Development Methods for Mobile Apps

Method	Description
Component-Based	Reusable UI components (same as React)
Offline-First	Store data locally, sync when online
Push Notifications	Engage users with timely updates
Deep Linking	Navigate directly to specific content from links
Biometric Auth	Fingerprint/Face ID for security
Infinite Scroll	Load more data as user scrolls
Pull to Refresh	Common mobile pattern for refreshing content
Responsive Layout	Adapt to different screen sizes and orientations

4 Essential Tools for Mobile Development

4.1 React Native Development Environment

- **Node.js:** JavaScript runtime
- **Expo:** Fastest way to start React Native (recommended for beginners)
- **React Native CLI:** More control, native modules
- **Android Studio:** For Android emulator and builds
- **Xcode:** For iOS emulator and builds (Mac only)
- **Expo Go App:** Test on physical device instantly

4.2 React Native Libraries

Library	Purpose
React Navigation	Stack, tab, drawer navigation
React Native Paper	Material Design components
NativeWind	Tailwind CSS for React Native
React Native Async Storage	Local persistent storage
React Native Vector Icons	Icon library
React Native Image Picker	Camera and gallery access

React Native Geolocation	GPS location services
React Native Maps	Google Maps/Apple Maps integration
React Native Firebase	Backend services (auth, database, storage)
React Native Push Notification	Push notifications
Axios	API calls
React Native Reanimated	Advanced animations
React Native Gesture Handler	Touch gestures

4.3 Backend for Mobile Apps

- **Node.js + Express:** Same as web apps (reuse your skills!)
- **Firebase:** Backend-as-a-service (auth, database, storage, push notifications)
- **Supabase:** Open-source Firebase alternative
- **MongoDB Realm:** Sync data offline to online

4.4 Testing & Debugging

- **React Native Debugger:** Standalone debugger
- **Flipper:** Mobile app debugger from Meta
- **Expo Go:** Test on real device instantly
- **Reactotron:** Debug React Native apps

4.5 Building & Distribution

- **EAS Build (Expo):** Cloud builds for iOS and Android
- **Android:** Generate APK/AAB, publish to Google Play Store
- **iOS:** Generate IPA, publish to Apple App Store (requires Apple Developer account)
- **TestFlight:** Beta testing for iOS
- **Google Play Beta:** Beta testing for Android

5 Learning Resources

Resource	Focus
React Native Official Docs	https://reactnative.dev
Expo Documentation	https://docs.expo.dev
React Navigation Docs	Navigation patterns
FreeCodeCamp - React Native	5-hour video course
Net Ninja React Native	YouTube playlist
CodeWithMosh	React Native course

6 Project Sequence for Mobile Apps

6.1 Level 1 – Basic React Native Apps

1. **To-Do List App:** Add, complete, delete tasks
2. **Counter App:** State management basics
3. **Calculator:** UI layout and event handling
4. **Weather App:** API calls, permissions

6.2 Level 2 – Navigation & Multi-Screen Apps

1. **Blog Reader:** List of posts → detail screen
2. **Recipe App:** Categories → recipes → details
3. **E-commerce Product Browser:** Search, filter, product details
4. **Profile App:** Tab navigation, user settings

6.3 Level 3 – Device Features Integration

1. **Camera App:** Take photos, save to gallery
2. **Location Tracker:** GPS, maps, location sharing
3. **QR Code Scanner:** Scan, generate QR codes
4. **Contact Manager:** Device contacts integration

6.4 Level 4 – Full-Stack Mobile Apps

1. **Authentication App:** Login/register with your Node.js backend
2. **Social Media Feed:** Posts, likes, comments from API
3. **Chat App:** Real-time messaging with Socket.io
4. **Expense Tracker:** Sync data with cloud database

6.5 Level 5 – Production-Ready Apps

1. **Offline-First Notes App:** Local storage + cloud sync
2. **Push Notification App:** Firebase Cloud Messaging
3. **Fitness Tracker:** Step counter, workout logger, charts
4. **Complete E-commerce App:** Cart, checkout, payments, order history

7 Concentration: React Native Deep Dive

7.1 Core Components vs Web

Web	React Native	Purpose
div	View	Container
span, p	Text	Display text
button	Button, TouchableOpacity	Touch interactions
input	TextInput	Text input
img	Image	Images
ul, li	FlatList	Scrolling lists
scroll	ScrollView	Scrollable container
CSS	StyleSheet	Styling

7.2 Basic React Native Component

Listing 1: First React Native Component

```
1 import React, { useState } from 'react';
2 import {
3   View,
4   Text,
5   TextInput,
6   TouchableOpacity,
7   FlatList,
8   StyleSheet
9 } from 'react-native';
10
11 const TodoApp = () => {
12   const [todos, setTodos] = useState([]);
```

```
13   const [text, setText] = useState('');
14
15   const addTodo = () => {
16     if (text.trim()) {
17       setTodos([...todos, { id: Date.now(), text, completed: false }]);
18       setText('');
19     }
20   };
21
22   const toggleTodo = (id) => {
23     setTodos(todos.map(todo =>
24       todo.id === id ? { ...todo, completed: !todo.completed } : todo
25     ));
26   };
27
28   const renderItem = ({ item }) => (
29     <TouchableOpacity onPress={() => toggleTodo(item.id)}>
30       <View style={styles.todoItem}>
31         <Text style={item.completed ? styles.completed : styles.
32           todoText}>
33           {item.text}
34         </Text>
35       </View>
36     </TouchableOpacity>
37   );
38
39   return (
40     <View style={styles.container}>
41       <Text style={styles.title}>My Todos</Text>
42       <View style={styles.inputContainer}>
43         <TextInput
44           style={styles.input}
45           value={text}
46           onChangeText={setText}
47           placeholder="Add a new todo..."
48         />
49       <TouchableOpacity style={styles.button} onPress={addTodo}>
50         <Text style={styles.buttonText}>Add</Text>
51       </TouchableOpacity>
52     </View>
53     <FlatList
54       data={todos}
55       renderItem={renderItem}
56       keyExtractor={(item) => item.id.toString()}
57     />
58   </View>
59 );
60
61 const styles = StyleSheet.create({
62   container: {
63     flex: 1,
64     padding: 20,
65     backgroundColor: '#fff'
66   },
```



```
67   title: {
68     fontSize: 24,
69     fontWeight: 'bold',
70     marginBottom: 20
71   },
72   inputContainer: {
73     flexDirection: 'row',
74     marginBottom: 20
75   },
76   input: {
77     flex: 1,
78     borderWidth: 1,
79     borderColor: '#ddd',
80     padding: 10,
81     borderRadius: 5,
82     marginRight: 10
83   },
84   button: {
85     backgroundColor: '#007AFF',
86     padding: 10,
87     borderRadius: 5,
88     justifyContent: 'center'
89   },
90   buttonText: {
91     color: '#fff',
92     fontWeight: 'bold'
93   },
94   todoItem: {
95     padding: 15,
96     borderBottomWidth: 1,
97     borderBottomColor: '#ddd'
98   },
99   todoText: {
100     fontSize: 16
101   },
102   completed: {
103     fontSize: 16,
104     textDecorationLine: 'line-through',
105     color: '#888'
106   }
107 });
108
109 export default TodoApp;
```

7.3 Navigation with React Navigation

Listing 2: Stack and Tab Navigation

```
1 // App.js
2 import { NavigationContainer } from '@react-navigation/native';
3 import { createNativeStackNavigator } from '@react-navigation/native-stack';
4 import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';
5 import Ionicons from 'react-native-vector-icons/Ionicons';
```

```
6
7 // Screens
8 import HomeScreen from './screens/HomeScreen';
9 import ProfileScreen from './screens/ProfileScreen';
10 import SettingsScreen from './screens/SettingsScreen';
11 import LoginScreen from './screens/LoginScreen';
12
13 const Stack = createNativeStackNavigator();
14 const Tab = createBottomTabNavigator();
15
16 // Tab Navigator (for authenticated users)
17 function MainTabs() {
18   return (
19     <Tab.Navigator
20       screenOptions={({ route }) => ({
21         tabBarIcon: ({ focused, color, size }) => {
22           let iconName;
23           if (route.name === 'Home') iconName = focused ? 'home' : '
             home-outline';
24           else if (route.name === 'Profile') iconName = focused ? '
             person' : 'person-outline';
25           else if (route.name === 'Settings') iconName = focused ? '
             settings' : 'settings-outline';
26           return <Ionicons name={iconName} size={size} color={color}
             />;
27         },
28         tabBarActiveTintColor: '#007AFF',
29         tabBarInactiveTintColor: 'gray'
30       })}
31     >
32       <Tab.Screen name="Home" component={HomeScreen} />
33       <Tab.Screen name="Profile" component={ProfileScreen} />
34       <Tab.Screen name="Settings" component={SettingsScreen} />
35     </Tab.Navigator>
36   );
37 }
38
39 // Main App with Auth Flow
40 export default function App() {
41   const [isLoggedIn, setIsLoggedIn] = useState(false);
42
43   return (
44     <NavigationContainer>
45       <Stack.Navigator>
46         {!isLoggedIn ? (
47           <Stack.Screen name="Login" component={LoginScreen} />
48         ) : (
49           <Stack.Screen name="Main" component={MainTabs} options={{
             headerShown: false }} />
50         )}
51       </Stack.Navigator>
52     </NavigationContainer>
53   );
54 }
55
```

```
56 // Example of navigation between screens
57 function HomeScreen({ navigation }) {
58   return (
59     <View>
60       <Button
61         title="Go to Details"
62         onPress={() => navigation.navigate('Details', { itemId: 123 })}
63       />
64     </View>
65   );
66 }
67
68 function DetailsScreen({ route, navigation }) {
69   const { itemId } = route.params;
70   return (
71     <View>
72       <Text>Item ID: {itemId}</Text>
73       <Button title="Go Back" onPress={() => navigation.goBack()} />
74     </View>
75   );
76 }
```

7.4 API Calls from React Native

Listing 3: Fetching Data from Backend

```
1 // services/api.js
2 import axios from 'axios';
3 import AsyncStorage from '@react-native-async-storage/async-storage';
4
5 const API_URL = 'https://your-backend.com/api';
6
7 const api = axios.create({
8   baseURL: API_URL,
9   headers: { 'Content-Type': 'application/json' }
10 });
11
12 // Add token to every request
13 api.interceptors.request.use(async (config) => {
14   const token = await AsyncStorage.getItem('token');
15   if (token) {
16     config.headers.Authorization = `Bearer ${token}`;
17   }
18   return config;
19 });
20
21 // Handle token expiration
22 api.interceptors.response.use(
23   (response) => response,
24   async (error) => {
25     if (error.response?.status === 401) {
26       // Token expired, logout user
27       await AsyncStorage.removeItem('token');
28       // Navigate to login (handle with event emitter or context)
29     }
29   }
29 );
```

```
30     return Promise.reject(error);
31   }
32 );
33
34 export default api;
35
36 // hooks/useTodos.js
37 import { useState, useEffect } from 'react';
38 import api from '../services/api';
39
40 export const useTodos = () => {
41   const [todos, setTodos] = useState([]);
42   const [loading, setLoading] = useState(true);
43   const [error, setError] = useState(null);
44
45   const fetchTodos = async () => {
46     try {
47       setLoading(true);
48       const response = await api.get('/todos');
49       setTodos(response.data);
50     } catch (err) {
51       setError(err.message);
52     } finally {
53       setLoading(false);
54     }
55   };
56
57   const addTodo = async (title) => {
58     try {
59       const response = await api.post('/todos', { title });
60       setTodos([...todos, response.data]);
61     } catch (err) {
62       setError(err.message);
63     }
64   };
65
66   const updateTodo = async (id, updates) => {
67     try {
68       const response = await api.put(`/todos/${id}`, updates);
69       setTodos(todos.map(todo => todo.id === id ? response.data : todo)
70       );
71     } catch (err) {
72       setError(err.message);
73     }
74   };
75
76   const deleteTodo = async (id) => {
77     try {
78       await api.delete(`/todos/${id}`);
79       setTodos(todos.filter(todo => todo.id !== id));
80     } catch (err) {
81       setError(err.message);
82     }
83   };
84 }
```

```
84   useEffect(() => {
85     fetchTodos();
86   }, []);
87
88   return { todos, loading, error, addTodo, updateTodo, deleteTodo };
89 };
```

7.5 Device Features Integration

Listing 4: Camera and Image Picker

```
1 import React, { useState } from 'react';
2 import { View, Text, Image, Button, Alert } from 'react-native';
3 import * as ImagePicker from 'expo-image-picker';
4 import * as FileSystem from 'expo-file-system';
5
6 export default function CameraScreen() {
7   const [image, setImage] = useState(null);
8
9   // Request permissions
10  const requestPermissions = async () => {
11    const { status } = await ImagePicker.requestCameraPermissionsAsync(
12      );
13    if (status !== 'granted') {
14      Alert.alert('Permission needed', 'Camera permission is required')
15      ;
16      return false;
17    }
18    return true;
19  };
20
21  // Take photo with camera
22  const takePhoto = async () => {
23    const hasPermission = await requestPermissions();
24    if (!hasPermission) return;
25
26    const result = await ImagePicker.launchCameraAsync({
27      allowsEditing: true,
28      aspect: [4, 3],
29      quality: 0.8,
30      base64: true
31    });
32
33    if (!result.canceled) {
34      setImage(result.assets[0].uri);
35      // Upload to backend
36      await uploadImage(result.assets[0].uri);
37    }
38  };
39
40  // Pick from gallery
41  const pickImage = async () => {
42    const result = await ImagePicker.launchImageLibraryAsync({
43      mediaTypes: ImagePicker.MediaTypeOptions.Images,
44      allowsEditing: true,
```

```
43     quality: 0.8
44   });
45
46   if (!result.canceled) {
47     setImage(result.assets[0].uri);
48   }
49 };
50
51 // Upload image to backend
52 const uploadImage = async (uri) => {
53   const formData = new FormData();
54   formData.append('image', {
55     uri,
56     name: 'photo.jpg',
57     type: 'image/jpeg'
58   });
59
60   try {
61     const response = await api.post('/upload', formData, {
62       headers: { 'Content-Type': 'multipart/form-data' }
63     });
64     console.log('Upload success:', response.data);
65   } catch (error) {
66     console.error('Upload failed:', error);
67   }
68 };
69
70 return (
71   <View style={{ flex: 1, alignItems: 'center', justifyContent: 'center' }}>
72     {image && <Image source={{ uri: image }} style={{ width: 200, height: 200 }} />}
73     <Button title="Take Photo" onPress={takePhoto} />
74     <Button title="Pick from Gallery" onPress={pickImage} />
75   </View>
76 );
77 }
```

7.6 Location and Maps

Listing 5: GPS Location and Maps

```
1 import React, { useState, useEffect } from 'react';
2 import { View, Text, Button, StyleSheet } from 'react-native';
3 import MapView, { Marker } from 'react-native-maps';
4 import * as Location from 'expo-location';
5
6 export default function LocationScreen() {
7   const [location, setLocation] = useState(null);
8   const [errorMsg, setErrorMsg] = useState(null);
9
10  useEffect(() => {
11    (async () => {
12      // Request permission
```

```
13     const { status } = await Location.  
14       requestForegroundPermissionsAsync();  
15     if (status !== 'granted') {  
16       setErrorMsg('Permission to access location was denied');  
17       return;  
18     }  
19  
20     // Get current location  
21     const currentLocation = await Location.getCurrentPositionAsync  
22       ({});  
23     setLocation({  
24       latitude: currentLocation.coords.latitude,  
25       longitude: currentLocation.coords.longitude,  
26       latitudeDelta: 0.0922,  
27       longitudeDelta: 0.0421  
28     });  
29  
30     // Watch location changes  
31     useEffect(() => {  
32       const subscription = Location.watchPositionAsync(  
33         { accuracy: Location.Accuracy.High, timeInterval: 5000,  
34           distanceInterval: 10 },  
35         (newLocation) => {  
36           setLocation(prev => ({  
37             ...prev,  
38             latitude: newLocation.coords.latitude,  
39             longitude: newLocation.coords.longitude  
40           }));  
41         }  
42       );  
43  
44       return () => subscription.remove();  
45     }, []);  
46  
47     if (errorMsg) {  
48       return <Text>{errorMsg}</Text>;  
49     }  
50  
51     if (!location) {  
52       return <Text>Loading location...</Text>;  
53     }  
54  
55     return (  
56       <View style={styles.container}>  
57         <MapView style={styles.map} region={location}>  
58           <Marker coordinate={location} title="You are here" />  
59         </MapView>  
60       </View>  
61     );  
62   }  
63  
64   const styles = StyleSheet.create({  
65     container: { flex: 1 },
```

```
65   map: { flex: 1 }
66 });
```

7.7 Push Notifications with Expo

Listing 6: Push Notifications Setup

```
1 // App.js - Setup notifications on app start
2 import * as Notifications from 'expo-notifications';
3 import { registerForPushNotificationsAsync } from './utils/
  notifications';
4
5 Notifications.setNotificationHandler({
6   handleNotification: async () => ({
7     shouldShowAlert: true,
8     shouldPlaySound: true,
9     shouldSetBadge: true
10  })
11 });
12
13 export default function App() {
14   const [expoPushToken, setExpoPushToken] = useState('');
15
16   useEffect(() => {
17     registerForPushNotificationsAsync().then(token => {
18       setExpoPushToken(token);
19       // Send token to your backend
20       if (token) {
21         api.post('/users/push-token', { token });
22       }
23     });
24   }, []);
25
26   // Handle notification when app is in foreground
27   useEffect(() => {
28     const subscription = Notifications.addNotificationReceivedListener(
29       notification => {
30         console.log('Notification received:', notification);
31       });
32     return () => subscription.remove();
33   }, []);
34
35   // Handle notification tap (opens app)
36   useEffect(() => {
37     const subscription = Notifications.
38       addNotificationResponseReceivedListener(response => {
39         const data = response.notification.request.content.data;
40         // Navigate based on notification data
41         console.log('Notification tapped:', data);
42       });
43     return () => subscription.remove();
44   }, []);
45
46   return <MainApp />;
47 }
```



```
46
47 // utils/notifications.js
48 import * as Notifications from 'expo-notifications';
49 import * as Device from 'expo-device';
50 import { Platform } from 'react-native';
51
52 export async function registerForPushNotificationsAsync() {
53   let token;
54
55   if (Platform.OS === 'android') {
56     await Notifications.setNotificationChannelAsync('default', {
57       name: 'default',
58       importance: Notifications.AndroidImportance.MAX,
59       vibrationPattern: [0, 250, 250, 250],
60       lightColor: '#FF231F7C'
61     });
62   }
63
64   if (Device.isDevice) {
65     const { status: existingStatus } = await Notifications.
66       getPermissionsAsync();
67     let finalStatus = existingStatus;
68     if (existingStatus !== 'granted') {
69       const { status } = await Notifications.requestPermissionsAsync();
70       finalStatus = status;
71     }
72     if (finalStatus !== 'granted') {
73       alert('Failed to get push token for push notification!');
74       return;
75     }
76     token = (await Notifications.getExpoPushTokenAsync()).data;
77   } else {
78     alert('Must use physical device for Push Notifications');
79   }
80   return token;
81 }
```

8 Building and Publishing

8.1 Expo Build Commands

```
1 # Install EAS CLI
2 npm install -g eas-cli
3
4 # Login to Expo
5 eas login
6
7 # Configure build
8 eas build:configure
9
10 # Build for Android (APK for testing)
11 eas build -p android --profile preview
```

```
12
13 # Build for Android (AAB for Play Store)
14 eas build -p android --profile production
15
16 # Build for iOS (requires Mac)
17 eas build -p ios --profile production
18
19 # Submit to stores
20 eas submit -p android
21 eas submit -p ios
```

8.2 App Store Requirements

- **Android:** Google Play Developer account (\$25 one-time)
- **iOS:** Apple Developer account (\$99/year)
- App icon (1024x1024)
- Screenshots (minimum 2 per device type)
- App description and keywords
- Privacy policy URL
- Content rating questionnaire
- Test on multiple devices

9 Learning Plan for Mobile

Week	Focus
1-2	React Native basics: components, props, state, styling
3-4	Navigation: Stack, Tab, Drawer navigators
5-6	Lists: FlatList, SectionList, performance optimization
7-8	API integration: Axios, AsyncStorage, offline support
9-10	Device features: Camera, Location, Image Picker
11-12	Advanced: Animations, Gestures, Push Notifications
13-14	State management: Context API, Redux Toolkit
15-16	Testing and debugging
17-18	Build and publish to app stores
19-20	Complete portfolio projects